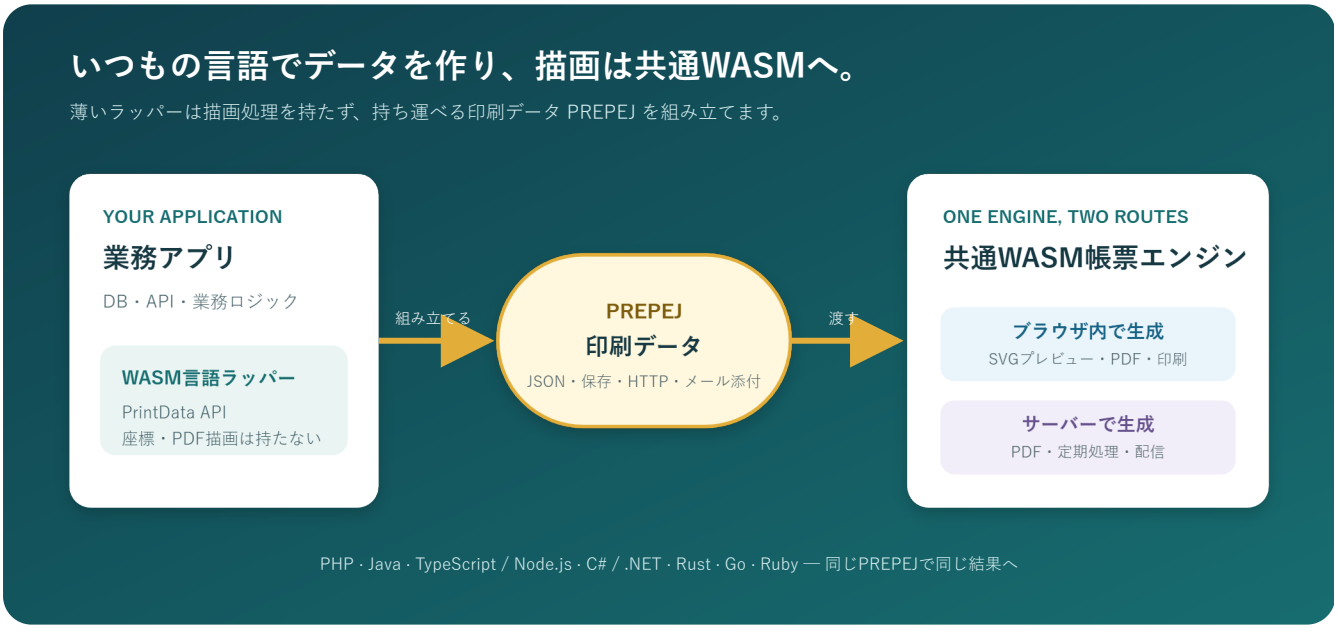
Reports.web Java WASM言語ラッパー プログラマーズガイド／APIリファレンス

Reports.webのWASM言語ラッパーは、Javaの業務データから、共通の印刷データPREPEJを組み立てるための小さなライブラリです。帳票の座標計算やPDF描画を各言語へ重複実装せず、ブラウザまたはサーバーの共通WASM帳票エンジンへ任せます。

この一冊では、初めて利用する方が次のプログラムを作れるようになることを目標にしています。

- Javaから帳票定義PREPDJへ業務データを流し込む
- 完成した印刷データPREPEJをJSON、ファイル、HTTPで受け渡す
- ブラウザでSVGプレビュー・印刷・PDF生成を行う
- サーバー側の共通WASMエンジンでPDFを生成・配信する
- Javaラッパーの主要APIを調べる



目次

1. 最初に知っておくこと 3

    1.1 「WASM言語ラッパー」とは 3

    1.2 帳票ができるまで 3

    1.3 二つのファイル 3

2. サンプルを動かす 3

3. 最初の一枚を作る 4

4. PrintData APIリファレンス 5

    4.1 値とインデックス 6

    4.2 ページごとに定義を切り替える 7

5. 明細と改ページ 7

6. 画像・バーコード・動的属性 8

7. ブラウザへ渡して表示する 8

8. PDFを作る二つの場所 9

    8.1 ブラウザで作る 9

    8.2 サーバーで作る 9

9. PREPEJを保存・再利用する 9

10. Web APIとデータベース 9

11. テストとリリース確認 10

12. よくある問題 10

13. 配置・更新・ライセンス 10

1. 最初に知っておくこと

1.1 「WASM言語ラッパー」とは

本書では、各言語でPREPEJを組み立てる補助ライブラリを**WASM言語ラッパー**と呼びます。実装の中心は`PrintData`です。ラッパー自身はWASMを直接ロードせず、SVGやPDFも描きません。出力形式を共通JSONに揃えることで、業務処理と描画エンジンを疎結合にします。

これはPure Java／Pure Python／WPF・.NETの言語別エンジンとは異なる方式です。Pure系は各言語の帳票エンジンを直接呼びます。WASM方式は7言語が同じWASMエンジンとブラウザUIを共有します。

1.2 帳票ができるまで

1. デザイナーで帳票定義 `.prepdj` を作ります。
2. Javaの`PrintData`へPREPDJを設定します。
3. DBやAPIから取得した値を、デザイン上のオブジェクト名へ設定します。
4. 1ページずつ確定し、`.prepej`を作ります。
5. PREPEJをブラウザのプレビュアー、またはサーバーWASMへ渡します。
6. 共通エンジンがSVGプレビュー、PDF、印刷を担当します。

1.3 二つのファイル

拡張子	呼び名	役割
`.prepdj`	帳票定義	用紙、文字、罫線、画像、バーコード、繰返しなどのデザイン
`.prepej`	印刷データ（Print Document）	帳票定義とページごとの値・属性をまとめた、再表示・再印刷可能な文書

PREPEJは単なる一時レスポンスではありません。ファイル保存、再読込、メール添付、REST API、サーバー間転送、後日の再印刷に同じ文書を使えます。

2. サンプルを動かす

基準環境は **Java 21** です。同梱のDocker構成では必要な言語環境とDBをコンテナ内に用意するため、まずPC側で準備するのはDockerです。Windows／MacではDocker Desktopを起動し、LinuxではDocker EngineとComposeプラグインが使える状態にしてください。

ZIPを全体ごと展開してください。共通RuntimeとDB初期化データも使うため、番号フォルダーだけを取り出さないでください。初回のイメージ取得とビルドにはインターネット接続が必要です。

短いコードから学ぶ場合は `WebAssembly/Samples/java/01.`あっという間に帳票出力`/README.html` から始めます。ここでは全帳票を確認できる02の起動を説明します。

1. VS CodeやCursor、または使い慣れたエディターで `WebAssembly/Samples/java/02.`全帳票を`Web`でプレビュー を開きます。
2. そのフォルダーでターミナルを開き、次を実行します。開発用のReports Webリポジトリをビルドする必要はありません。

```
docker compose up -d --build --wait
docker compose ps
```

3. 起動が成功してから、ブラウザで `http://127.0.0.1:19211/` を開きます。URLを開くだけではアプリは起動しません。
4. 終了するときは、同じフォルダーで `docker compose down` を実行します。このサンプルのコンテナとネットワークが削除されます。DBデータの初期化やイメージの片付けは、そのフォルダーのREADMEに手順があります。

02では、単純帳票、10の倍数、郵便番号一覧、見積書、請求書、商品一覧、名刺、デザイン機能見本を確認できます。請求書などを選び、ソースと出力結果を見比べてください。別のアプリから帳票APIを呼ぶ使い方は、続く03のREADMEに説明しています。

3. 最初の一枚を作る

01はJDK標準機能だけで印刷データのJSONを作る例です。PrintDataクラスを使う例は、必要なライブラリが揃った02のSpring Bootプロジェクトで試しましょう。02をREADMEに従って起動し、一度帳票を表示しておきます。

02の`sample-app/src/main/java/jp/pao/reports/sample/SampleController.java`をコピーして保管し、`public class SampleController {`の内側、既存メソッドと同じ階層へ、次のメソッドを追加します。既存コードを削除する必要はありません。

```
@GetMapping("/first-report")
@ResponseBody
ResponseEntity<String> firstReport() throws Exception {
    // 同梱の「単純帳票」の定義を読みます。DBからの取得は不要です。
    JsonNode definition = catalog.definition("quick-start");
    jp.pao.reports.web.PrintData print = new jp.pao.reports.web.PrintData()
        .setDefinition(definition)
        .pageStart()
        .setValue("Text2", "Javaで帳票出力")
        .pageEnd();
    return ResponseEntity.ok()
        .contentType(MediaType.APPLICATION_JSON)
        .header("Content-Disposition", "attachment; filename=first-report.prepej")
        .body(print.toJson());
}
```

02のフォルダーで`docker compose up -d --build --wait --wait-timeout 180`を実行して、変更したJavaをコンパイルします。成功したら`http://127.0.0.1:19211/demo/reports.web/samples/java/first-report`をブラウザーで開きます。ダウンロードした`first-report.prepej`を、02の帳票画面の「印刷データを開く」で読み込んでください。変更した文字が表示され、PDFにも出せます。元に戻すときは追加メソッドを削除し、同じコマンドで再ビルドします。

`Text2`は02の`resources/definitions/quick-start.prepdj`にある動的テキストの名前です。01の`Message`とは違います。自分の帳票では、読み込む定義とオブジェクト名を一緒に変更してください。`JsonNode`等のimportは既存コントローラーにあり、PrintDataの依存関係も02の`pom.xml`で設定済みです。上の例はそのプロジェクトへ追加するメソッドで、単独のJavaファイルではありません。

呼出順は **定義を設定 → ページを開始 → 値を設定 → ページを確定 → JSONを取得** です。未完了のページがある状態ではPREPEJを取得できません。この制約により、途中までの帳票を誤って配信することを防ぎます。

4. PrintData APIリファレンス

配布ZIP内の実装ファイルは `WebAssembly/Samples/java/02.全帳票をWebでプレビュー/report-data/src/main/java/jp/pao/reports/web/PrintData.java` です。各番号のサンプルにも必要なソースを置いています。次のAPI名は実装に合わせています。

API	用途
`setDefinition`	帳票定義（PREPDJ）を設定します。設定値は複製され、呼び出し元の変更から守られます。
`pageStart`	新しい印刷ページを開始します。別の定義を渡すと、表紙から明細などへ様式を切り替えられます。
`setValue`	名前で指定した動的オブジェクトへ値を設定します。indexは0から始まる繰返し位置です。
`setRepeatedValue`	縦横二方向に繰り返すオブジェクトへ、X・Y位置を指定して値を設定します。
`setTypedValue`	DynamicText、DynamicImage、DynamicBarcodeなど、値の種類を明示して設定します。
`setImage`	画像URLまたはdata URIを動的画像へ設定します。
`setBarcode`	バーコード／二次元コードへ渡す文字列を設定します。
`drawObject`	値を変更せず、指定オブジェクト／繰返し位置を描画対象にします。
`changeAttributes`	位置、幅、色、フォント、配置などをページ単位で動的に変更します。
`changeRepeatedAttributes`	繰返し位置を含めて動的属性を変更します。
`pageEnd`	現在のページを確定し、印刷データへ追加します。
`toObjectNode / toJson`	完成したPREPEJをオブジェクトまたはJSON文字列として取得します。
`save`	完成したPREPEJを一時ファイル経由で安全に保存します。

4.1 値とインデックス

`new PrintData()`は引数なしです。設定メソッドと`pageStart`／`pageEnd`は同じ`PrintData`を返すため、続けて呼べます。よく使う省略形と、すべての引数を指定する形を以下に示します。

呼び方	引数の意味・戻り値
`setDefinition(JsonNode definition)`	共通の帳票定義を設定
`pageStart()` / `pageStart(JsonNode definition)`	ページ開始。引数ありならそのページの定義を指定
`setValue(String name, Object value)`	動的文字の名前と値。位置0、描画あり
`setValue(name, value, int index)` / `setValue(name, value, int index, boolean drawing)`	繰返し位置と描画有無を指定
`setRepeatedValue(name, value, int x, int y)` / `setRepeatedValue(name, value, int x, int y, boolean drawing)`	二方向の繰返し位置を指定
`setTypedValue(String name, String type, Object value, int index, boolean drawing)`	型・位置・描画有無を明示
`setImage(String name, String source)` / `setImage(name, source, int index, boolean drawing)`	画像のURLまたはdata URI
`setBarcode(String name, String code)` / `setBarcode(name, code, int index, boolean drawing)`	バーコードの内容
`drawObject(String name)` / `drawObject(name, int index, String type)`	値を変えずに描画。型を省略する1引数版は位置0で、当該ページの定義からDynamic型を推論
`changeAttributes(String name, Map<String, ?> values)` / `changeAttributes(name, values, int index)`	属性名→値のMapで変更
`changeRepeatedAttributes(String name, Map<String, ?> values, int index, Integer x, Integer y, String type)`	二方向の位置と型も指定。xとyは両方指定、または両方null
`pageEnd()`	ページを確定
`toObjectNode()`	完成したJSONオブジェクト（Jackson ObjectNode）を返す
`toJson()` / `toJson(boolean pretty)`	JSON文字列。省略時は整形あり
`save(Path destination)`	UTF-8 JSONファイルへ保存。戻り値なし、失敗時IOException

これは引数を調べる一覧です。`name`は定義のオブジェクト名で、画面に見える文字そのものではありません。`Object`型の値も自由なJavaオブジェクトではなく、下記の単純値に限ります。`save()`はJavaが動く側のファイルへの保存です。Web利用者のPCへのダウンロードは、第3節のHTTP応答のように別途行います。保存先フ

ォルダーと書込権限を用意してください。

呼出順の違反は`IllegalStateException`、不正な値や位置は`IllegalArgumentException`です。帳票ごと／HTTP要求ごとに新しいPrintDataを作り、複数要求から同じインスタンスを共有しないでください。

値には文字列、数値、真偽値、nullの単純値を渡します。配列や任意オブジェクトを値として渡さず、帳票オブジェクトごとに明示的に対応付けてください。繰返しのindex、indexX、indexYは0から始まり、負数はエラーです。

`drawing=false`は、その繰返し位置を描画しない場合に使います。同じ名前・同じ位置への設定はサンプルの規約に従い、意図しない重複を作らないようにします。

型を省略するAPIは、そのページで有効な帳票定義から対応するDynamic型を推論します。型を明示したoverloadは指定値を優先します。定義にない名前は従来互換の`DynamicText`になります。空の名前、負のindex、x/yの片方だけの指定、null属性値、非有限の数値は例外です。

4.2 ページごとに定義を切り替える

表紙と本文、途中から異なる一覧形式などでは、ページ開始時に別のPREPDJを渡します。座標をプログラムへ直接書くのではなく、デザイナーで完成させた定義をページへ結び付けます。定義を変更する前に現在のページを必ず確定してください。

5. 明細と改ページ

以下は考え方を示す抜粋です。`definition`には「ページ・品番・品名・金額」の動的テキストを持つ定義、`details`には`code()`・`name()`・`amount()`を持つ`Detail`のリストを用意します。これらを用意せずに第3節へ貼り付けても動きません。同梱帳票で動く完成例は02の`SampleCatalog.java`の`estimate()`／`invoice()`を参照してください。

```
PrintData print = new PrintData().setDefinition(definition);
for (int offset = 0, page = 1; offset < details.size(); offset += 20, page++) {
    print.pageStart().setValue("ページ", page);
    for (int row = 0; row < Math.min(20, details.size() - offset); row++) {
        Detail d = details.get(offset + row);
        print.setValue("品番", d.code(), row)
             .setValue("品名", d.name(), row)
             .setValue("金額", d.amount(), row);
    }
    print.pageEnd();
}
```

1ページの行数は帳票定義と業務仕様から決めます。最終ページの空行を描画するか、明細ごとに罫線を出すかもプログラム側で明示します。ページ番号はPREPEJのページ順と、帳票へ表示する業務上のページ番号を分けて考えます。

6. 画像・バーコード・動的属性

次はページ開始と確定の間で呼ぶ抜粋です。「角印」は動的画像、「商品QR」は動的バーコード、「合計金額」は動的テキストとして定義し、`stampDataUri`には画像のdata URIを設定します。`Map`は`java.util.Map`です。

```
print.setImage("角印", stampDataUri)
     .setBarcode("商品QR", "https://example.jp/items/123")
     .changeAttributes("合計金額", Map.of(
         "bold", true, "fontSize", 16,
         "horizontalAlignment", "Right"));
```

画像は閲覧先でも取得できる管理下のURL、または許容サイズのdata URIにします。サーバーPDF生成では、任意の外部URLをそのまま取得するとSSRFにつながります。固定画像は許可リストまたはアプリ管理下のファイルから解決してください。

動的属性では、名前・型・値を無検証のリクエストから直接渡さないでください。変更できる属性をアプリ側で限定し、数値範囲、色、配置、フォント名を検証します。

7. ブラウザへ渡して表示する

Java APIはPREPEJをJSONで返し、ブラウザ側は共通のReports Web APIへ渡します。次は02の実際のURLを使った表示部分の抜粋です。完成した画面は02の`sample-app/src/main/resources/templates/index.html`を参照してください。

```
<iframe id="reportsViewer" title="帳票プレビュー" style="width:100%;height:80vh"
        src="/demo/reports.web/preview/?empty=1"></iframe>
<script>
const viewer = document.querySelector('#reportsViewer');
viewer.addEventListener('load', async () => {
  const response = await fetch('/demo/reports.web/samples/java/api?action=data&sample=invoice');
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  await viewer.contentWindow.ReportsWeb.setPrintData(await response.json());
});
</script>
```

親画面とプレビュアーは同一オリジンに置くのが最も簡単です。別オリジンで`postMessage`を使う場合は、送信元と送信先のoriginを固定し、`*`を本番環境で使わないでください。

プレビュアー側では`setZoom("auto")`、ページ移動、検索、印刷、`exportPdf()`、`savePrintData()`を利用できます。独自画面へ表示面だけを組み込む場合は`hideAll()`でツール群を隠し、同じ機能を外側のボタンから呼び出します。

8. PDFを作る二つの場所

8.1 ブラウザで作る

PREPEJを利用者のブラウザへ渡し、ブラウザ内のWASMがPDFを生成します。サーバーのPDF描画負荷を抑え、プレビューからそのまま保存・印刷する操作に向きます。機密データをブラウザへ渡してよいかは、業務の認可設計で判断します。

8.2 サーバーで作る

Spring Bootなどから共通Node/WASMサービスへHTTPでPREPEJを送ります。 定期処理、メール添付、保存、監査、画面を開かない帳票生成に向きます。

```
POST /render/pdf
Content-Type: application/json

<PREPEJ JSON>
```

成功時は`application/pdf`を返します。入力・出力サイズ、同時実行数、処理時間を制限し、失敗時の途中PDFを保存・配信しないでください。ブラウザ生成とサーバー生成は同じPREPEJを入力にするため、業務データ組立てを二重実装する必要はありません。

9. PREPEJを保存・再利用する

`save`は完成したPREPEJをUTF-8 JSONとして保存します。保存前にページをすべて確定してください。ファイル名には`.prepej`を使い、アップロード時には拡張子だけで信用せず、Format、Version、Definition、Pages、サイズ、ページ数を検証します。

保存した文書は、Webプレビュアーの「開く」またはドラッグ＆ドロップで再表示できます。HTTPで受け渡す場合もファイルと同じJSONです。再印刷の証跡として保管する場合は、業務データの保持期間、暗号化、アクセス権、削除規則を定めます。

10. Web APIとデータベース

DB列と帳票オブジェクト名はコードで明示的に対応付けます。SQLはプレースホルダーを使い、利用者が参照できる伝票かを取得前に確認します。日付、通貨、端数処理は帳票へ渡す前に業務規則で確定します。

機密帳票のレスポンスには`Cache-Control: no-store`を設定します。内部例外、接続文字列、ファイルパスを利用者へ返さず、要求IDを安全なサーバーログへ残します。帳票定義や画像は可能なら公開ルート外へ置きます。

11. テストとリリース確認

同梱02のDockerビルドは`mvn verify`を実行し、Javaの単体試験も行います。JDKとMavenを自分のPCに用意した場合は、02のフォルダーで`mvn -pl report-data test`を実行してラッパー単体を確認できます。画面操作とソース変更の確認は01〜03のREADMEに沿って行います。

最低限、次を確認します。

- Formatが`Reports.net PrintData`、Versionが`1.0`である
- PREPDJを設定せずにページを開始するとエラーになる
- ページを閉じずに次のページを開始できない
- 日本語、改行、数値、真偽値、nullが壊れない
- 明細の先頭・最終行、改ページ前後に欠落や重複がない
- ページ途中の帳票定義切替が、完了済みページを変更しない
- 画像・角印、バーコード、動的属性がSVGとPDFへ反映される
- 保存したPREPEJを開き直すと同じページ・内容になる
- ブラウザ生成とサーバー生成のページ数・主要表示が一致する

ラッパー変更時はWASM描画エンジン全体を毎回再試験するのではなく、生成PREPEJが共通仕様と基準サンプルに一致することを中心に確認します。WASM、プレビュアー、フォント、画像処理を変更した場合は、別途描画回帰試験が必要です。

12. よくある問題

症状	確認すること
帳票が空になる	PREPDJ上のオブジェクト名と`setValue`の名前、indexを確認します。
2ページ目がない	ページごとにpage start／page endが対になっているか確認します。
画像が×になる	URLの到達性、data URI、CORS、サーバー側許可リストを確認します。
日本語が欠ける	配置したフォント、フォントマップ、WASMとWeb資源の版を確認します。
WASMが読み込めない	`.wasm`のURL、`application/wasm`、キャッシュ、同一リリースのJSを確認します。
PDFだけ失敗する	入力サイズ、画像サイズ、タイムアウト、同時実行制限、サーバーログの要求IDを確認します。

13. 配置・更新・ライセンス

ラッパー、プレビュアー、JavaScript／TypeScript、CSS、WASMは同じReports.web配布版から取り出してください。HTMLだけ、またはラッパーだけを別バージョンへ差し替えると、PREPEJや公開APIの互換性を判断できなくなります。

現行の製品版への切替は、購入者のメールアドレスを含む電子署名付きの `reports-web.license` を、実際に帳票を出力する側へ配置して行います。購入者ごとにWASMを差し替える手順ではありません。ブラウザーとサーバーでは配置先が異なるため、製品版を使い始めるに従ってください。ライセンスファイルを公開サンプルやソース管理へ混入させないでください。正式な動作環境、使用許諾、更新履歴、問い合わせ先は共通製品マニュアルを正本とします。開発ライセンスは開発用PC 1台につき1ライセンスです。